

**LAPORAN PRAKTIKUM
APLIKASI BERBASIS PLATFORM
MODUL 1 – 3**



**Disusun oleh:
Titanio Francy Naddiansa
2311102289
S1 IF-11-04**

**Dosen Pengampu:
Cahyo Prihantoro, S.Kom., M.Eng.**

**PROGRAM STUDI S1 INFORMATIKA
FAKULTAS INFORMATIKA
TELKOM UNIVERSITY PURWOKERTO
2025/2026**

BAB 1 – RUNNING MODUL

(Instalasi dan Konfigurasi Lingkungan Pengembangan)

Tujuan Praktikum

Setelah menyelesaikan praktikum modul ini, mahasiswa diharapkan mampu:

- Melakukan instalasi dan konfigurasi Git sebagai sistem pengontrol versi pada proyek perangkat lunak.
- Melakukan instalasi JDK (Java Development Kit) serta mengatur environment variable agar dapat dikenali oleh sistem.
- Melakukan instalasi Flutter SDK dan mengintegrasikannya ke dalam PATH sistem operasi.
- Melakukan instalasi Android Studio beserta seluruh komponen SDK yang diperlukan untuk pengembangan aplikasi Android.
- Melakukan instalasi Visual Studio Code beserta ekstensi Flutter dan Dart sebagai code editor utama.

1.1. Git

Git merupakan salah satu sistem pengontrol versi (Version Control System / VCS) yang paling banyak digunakan dalam dunia pengembangan perangkat lunak. Diciptakan oleh Linus Torvalds, Git bertugas merekam setiap perubahan pada file-file di dalam sebuah proyek, baik yang dikerjakan secara individu maupun tim. Karena bersifat distributed revision control, repositori Git tidak hanya tersimpan di satu lokasi, melainkan dapat didistribusikan ke banyak mesin sekaligus.

Langkah-langkah instalasi Git pada sistem operasi Windows adalah sebagai berikut:

1. Unduh installer Git melalui <https://git-scm.com/download/win>.
2. Klik dua kali pada file installer yang telah diunduh untuk memulai proses instalasi.
3. Baca informasi lisensi yang ditampilkan, kemudian klik Next untuk melanjutkan.
4. Pilih komponen-komponen yang ingin dipasang sesuai kebutuhan, lalu klik Next.
5. Pilih editor teks default yang akan digunakan oleh Git. Pilih Nano untuk kemudahan penggunaan atau Vim apabila sudah terbiasa.
6. Pada bagian PATH environment, pilih opsi ketiga agar Command Prompt dapat mengenali perintah Git sekaligus beberapa perintah UNIX dasar.
7. Biarkan pengaturan lainnya pada nilai default hingga tahap instalasi, kemudian klik Install.
8. Verifikasi keberhasilan instalasi dengan menjalankan perintah berikut pada Command Prompt:

```
git --version
```

9. Setelah instalasi berhasil, lakukan konfigurasi identitas pengguna dengan perintah berikut:

```
git config --global user.name "Nama Anda"
```

```
git config --global user.email email.anda@contoh.com
```

Konfigurasi ini penting agar setiap commit yang dibuat tercatat atas nama pengguna yang benar.

1.2. Instalasi JDK

JDK (Java Development Kit) adalah perangkat wajib yang diperlukan untuk mengompilasi kode sumber Java menjadi bytecode yang kemudian dapat dieksekusi oleh JRE (Java Runtime Environment). Dalam ekosistem Flutter, JDK digunakan oleh Android Studio sebagai dasar kompilasi proyek Android. Langkah-langkah instalasinya adalah sebagai berikut:

10. Buka halaman unduhan JDK di <https://www.oracle.com/java/technologies/javase-jdk15-downloads.html>, kemudian pilih versi installer yang sesuai dengan sistem operasi yang digunakan.
11. Jalankan file installer yang telah diunduh dan klik Next pada jendela awal instalasi.
12. Pilih direktori instalasi JDK. Sangat disarankan menggunakan path default yang telah disediakan agar tidak terjadi konflik, lalu klik Next.
13. Tunggu hingga proses instalasi selesai, kemudian klik Close.
14. Buka direktori tempat JDK terinstal dan salin path-nya (contoh: C:\Program Files\Java\jdk-15).
15. Buka Environment Variables melalui menu Search di Start, ketik 'Environment Variables', lalu pilih Edit the system environment variables.
16. Pada jendela System Properties, pilih tab Advanced kemudian klik tombol Environment Variables.
17. Di bagian User Variables, klik New dan buat variabel baru dengan nama JAVA_HOME serta isi nilainya dengan path JDK yang telah disalin sebelumnya.
18. Ulangi langkah yang sama pada bagian System Variables agar JDK dikenali secara global oleh sistem.
19. Klik OK pada semua jendela yang terbuka untuk menyimpan perubahan.

1.3. Instalasi Flutter SDK

Flutter SDK adalah perangkat inti yang digunakan untuk mengembangkan aplikasi lintas platform. Sebelum memulai instalasi, pastikan perangkat yang digunakan memenuhi persyaratan minimum yang ditetapkan.

1.3.1. Persyaratan Minimum

a. Windows

- Sistem operasi Windows 7 SP1 atau lebih baru (64-bit), berbasis x86-64.
- Ruang penyimpanan minimal 1.64 GB, belum termasuk IDE dan tools tambahan.
- Windows PowerShell versi 5.0 atau lebih baru (telah tersedia secara bawaan di Windows 10).
- Git for Windows 2.x dengan opsi "Use Git from the Windows Command Prompt" diaktifkan.

b. macOS

- Sistem operasi macOS 64-bit.
- Ruang penyimpanan minimal 2.8 GB.

- Command-line tools yang wajib tersedia: bash, curl, git 2.x, mkdir, rm, unzip, dan which.

c. Linux

- Sistem operasi Linux 64-bit.
- Ruang penyimpanan minimal 1.8 GB.
- Command-line tools yang wajib tersedia: bash, curl, git 2.x, mkdir, rm, unzip, which, dan xz-utils.

1.3.2. Langkah Instalasi Flutter SDK

20. Unduh Flutter SDK versi stabil terbaru yang sesuai dengan sistem operasi melalui <https://flutter.dev/docs/development/tools/sdk/releases>.
21. Ekstrak file ZIP hasil unduhan dan letakkan folder flutter di lokasi yang diinginkan, misalnya C:\Development. Perhatian: hindari meletakkan Flutter di C:\Program Files atau direktori yang memerlukan hak akses administrator.
22. Masuk ke folder flutter yang telah diekstrak, kemudian klik dua kali pada flutter_console.bat untuk membuka Flutter Console.
23. Tambahkan path flutter\bin ke dalam variabel PATH pada Environment Variables, agar perintah Flutter dapat dijalankan dari terminal manapun.
24. Verifikasi instalasi Flutter dengan menjalankan perintah berikut pada terminal:

```
flutter doctor
```

Perintah flutter doctor akan memindai seluruh dependensi yang dibutuhkan Flutter, seperti Android SDK, iOS toolchain (khusus macOS), koneksi perangkat, dan lain-lain. Setiap item yang menampilkan tanda centang hijau berarti sudah terkonfigurasi dengan benar. Apabila terdapat item dengan tanda silang atau seru, ikuti petunjuk yang diberikan untuk menyelesaikan konfigurasi tersebut.

1.4. Instalasi Android Studio

Android Studio adalah Integrated Development Environment (IDE) resmi yang dirancang khusus untuk pengembangan aplikasi Android. IDE ini dilengkapi dengan emulator, debugger, dan berbagai tools pendukung yang memudahkan proses development.

1.4.1. Persyaratan Minimum

- Sistem operasi Microsoft Windows 7/8/10 (64-bit).
- RAM minimal 4 GB, direkomendasikan 8 GB untuk performa optimal.
- Ruang disk minimal 2 GB, direkomendasikan 4 GB (terdiri dari 500 MB untuk IDE dan 1.5 GB untuk Android SDK beserta system image emulator).

1.4.2. Proses Instalasi

Unduh Android Studio versi terbaru dari <https://developer.android.com/studio> kemudian ikuti langkah-langkah berikut:

25. Jalankan file installer (.exe) dengan cara klik dua kali.
26. Klik Next pada jendela sambutan instalasi.
27. Pastikan opsi Android Virtual Device (AVD) dicentang agar emulator dapat digunakan untuk pengujian aplikasi, kemudian klik Next.

28. Pilih direktori instalasi yang diinginkan, lalu klik Next.
29. Tentukan folder Start Menu untuk shortcut Android Studio, kemudian klik Install.
30. Tunggu hingga proses instalasi selesai sepenuhnya, lalu klik Next dan Finish.
31. Pada peluncuran pertama, pilih opsi "Do not import settings" jika ini adalah instalasi pertama.
32. Pilih Installation Type "Standard" (direkomendasikan) untuk mengunduh semua komponen penting secara otomatis, lalu klik Next.
33. Pilih tema tampilan IDE (Light atau Dark) sesuai preferensi, kemudian klik Next.
34. Tinjau daftar komponen yang akan diunduh, lalu klik Finish untuk memulai proses pengunduhan.
35. Setelah semua komponen terunduh, klik Finish. Android Studio siap digunakan.
36. Buat proyek baru dengan memilih "New Project", pilih template "Empty Activity".
37. Isi nama aplikasi, pilih bahasa pemrograman Java, lalu klik Finish.

1.5. Instalasi SDK Android

Android SDK (Software Development Kit) adalah kumpulan lengkap dari library, tools, platform, dan API yang dibutuhkan untuk mengembangkan aplikasi Android. SDK ini wajib terinstal di dalam Android Studio. Berikut cara menginstalnya:

- Pada menu bar Android Studio, klik menu Tools kemudian pilih SDK Manager.
- Di tab SDK Platform, centang versi Android yang ingin digunakan sebagai target pengembangan.
- Pindah ke tab SDK Tools, centang tools tambahan yang diperlukan seperti Android SDK Build-Tools, Android Emulator, dan lainnya.
- Klik OK atau Apply, lalu setuju License Agreement untuk setiap komponen yang akan diunduh.
- Tunggu proses pengunduhan dan instalasi komponen hingga selesai, kemudian klik Finish.

1.6. Instalasi Visual Studio Code

Visual Studio Code (VS Code) adalah code editor sumber terbuka yang ringan namun sangat bertenaga, dikembangkan oleh Microsoft. VS Code mendukung banyak bahasa pemrograman melalui sistem ekstensi dan sangat populer di kalangan pengembang Flutter karena kemudahan penggunaannya. Cara instalasinya:

38. Unduh VS Code dari <https://code.visualstudio.com/Download>, pilih versi yang sesuai dengan sistem operasi.
39. Jalankan installer yang telah diunduh.
40. Ikuti petunjuk instalasi, biarkan semua pengaturan pada nilai default.
41. Klik Next dan Install, kemudian tunggu hingga proses selesai.
42. Klik Finish. VS Code akan terbuka secara otomatis setelah instalasi.

1.7. Instalasi Extension Visual Studio Code

Untuk dapat mengembangkan aplikasi Flutter menggunakan VS Code, dua ekstensi berikut wajib dipasang terlebih dahulu:

a. Ekstensi Dart

Ekstensi Dart diperlukan karena Flutter dibangun di atas bahasa pemrograman Dart. Ekstensi ini menyediakan fitur syntax highlighting, code completion, dan debugging untuk kode Dart.

b. Ekstensi Flutter

Ekstensi Flutter menambahkan dukungan penuh untuk pengembangan aplikasi Flutter di dalam VS Code, termasuk hot reload, widget inspector, dan perintah-perintah Flutter langsung dari Command Palette.

Langkah instalasi kedua ekstensi tersebut:

- Buka VS Code, kemudian klik ikon Extensions pada sidebar atau tekan Ctrl+Shift+X.
- Pada kotak pencarian, ketik "Flutter" kemudian pilih ekstensi Flutter by Dart Code dan klik Install. Ekstensi Dart akan ikut terpasang secara otomatis.
- Verifikasi instalasi dengan membuka Command Palette (Ctrl+Shift+P) dan menetik "Flutter: New Project". Jika opsi tersebut muncul, maka instalasi telah berhasil.

BAB 2 – PENGENALAN FLUTTER

Tujuan Praktikum

- Mahasiswa mampu memahami konsep dasar dan arsitektur Flutter sebagai framework pengembangan aplikasi lintas platform.
- Mahasiswa dapat mengimplementasikan struktur widget dasar Flutter dan membuat aplikasi Hello World pertama.

2.1. Apa Itu Flutter

Flutter adalah framework pengembangan aplikasi lintas platform (cross-platform) yang dikembangkan oleh Google. Dengan Flutter, pengembang cukup menulis kode sekali namun dapat menjalankan aplikasinya di berbagai platform seperti Android, iOS, Web, Desktop (Windows, macOS, Linux), sekaligus.

Secara teknis, Flutter ditulis menggunakan bahasa C, C++, dan Dart, serta mengandalkan Google's Skia Graphics Engine untuk me-render antarmuka pengguna. Engine rendering yang sama ini juga digunakan oleh produk-produk besar seperti Google Chrome, Chrome OS, Mozilla Firefox, dan Android itu sendiri, sehingga Flutter mampu menghasilkan tampilan yang konsisten di berbagai platform.

Flutter berjalan di atas Dart Virtual Machine (VM) pada sistem operasi Windows, Linux, dan macOS. Dart VM mendukung mode kompilasi Just-in-Time (JIT) selama proses pengembangan, yang memungkinkan fitur hot-reload — yakni kemampuan untuk melihat perubahan kode secara langsung tanpa perlu me-restart aplikasi. Hal ini sangat mempercepat siklus pengembangan.

Konsep utama dalam Flutter adalah widget. Semua elemen tampilan — mulai dari tombol, teks, gambar, hingga layout — direpresentasikan sebagai widget. Widget-widget ini tersusun dalam sebuah hierarki yang disebut widget tree, di mana setiap widget dapat mengandung widget lainnya. Terdapat dua jenis widget utama: StatelessWidget (tidak memiliki state yang berubah) dan StatefulWidget (memiliki state yang dapat berubah seiring interaksi pengguna).

2.2. Arsitektur Flutter

Flutter menggunakan pendekatan arsitektur berlapis (layered architecture). Dari lapisan paling bawah hingga atas terdiri dari: Embedder Layer (kode platform-spesifik), Engine Layer (C/C++ + Skia untuk rendering), Framework Layer (Dart SDK: widgets, material, cupertino), dan di paling atas adalah Application Layer (kode aplikasi yang ditulis oleh developer).

Untuk pengelolaan state, Flutter merekomendasikan penggunaan arsitektur Business Logic Component (BLoC). Arsitektur BLoC memisahkan logika bisnis dari lapisan tampilan (UI) dengan memanfaatkan konsep Streams. Cara kerjanya sederhana: ketika terjadi suatu event (misalnya pengguna menekan tombol), event tersebut dikirimkan ke BLoC, BLoC memproses logika yang diperlukan, lalu mengeluarkan state baru yang akan digunakan oleh widget untuk memperbarui tampilan.

Tiga keunggulan utama arsitektur BLoC adalah:

- Simplicity (Kesederhanaan) — alur data yang jelas dan mudah dipahami karena dipisahkan antara UI dan logika.
- Scalability (Skalabilitas) — memudahkan pengembangan fitur baru tanpa mengganggu kode yang sudah ada.
- Testability (Kemampuan Pengujian) — logika bisnis yang terpisah dari UI dapat diuji secara independen.

2.3. Hello World Pada Flutter

Berikut adalah langkah-langkah lengkap membuat aplikasi Hello World pertama menggunakan Flutter dan VS Code:

43. Buka VS Code dan pastikan ekstensi Flutter serta Dart sudah terpasang dengan benar.
44. Buat proyek Flutter baru melalui View → Command Palette → ketik Flutter: New Project → pilih Application → tentukan folder penyimpanan → beri nama project (misalnya hello_world) → tekan Enter dan tunggu hingga proses pembuatan selesai.
45. Setelah proyek selesai dibuat, buka file lib/main.dart pada panel explorer di sisi kiri.
46. Hapus seluruh kode bawaan yang ada di dalam main.dart.
47. Ketikkan kode berikut:

```
// ignore_for_file: prefer_const_constructors
import 'package:flutter/material.dart';
```

```
void main() {
  runApp(const MyApp());
}
```

```
class MyApp extends StatelessWidget {
  const MyApp({Key? key}) : super(key: key);
```

```
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Hello World',
      home: const MyHomePage(title: 'Flutter Hello World Page'),
    );
  }
}
```

```
class MyHomePage extends StatefulWidget {
  const MyHomePage({Key? key, required this.title}) : super(key: key);
  final String title;
```

```
  @override
  State<MyHomePage> createState() => _MyHomePageState();
}
```

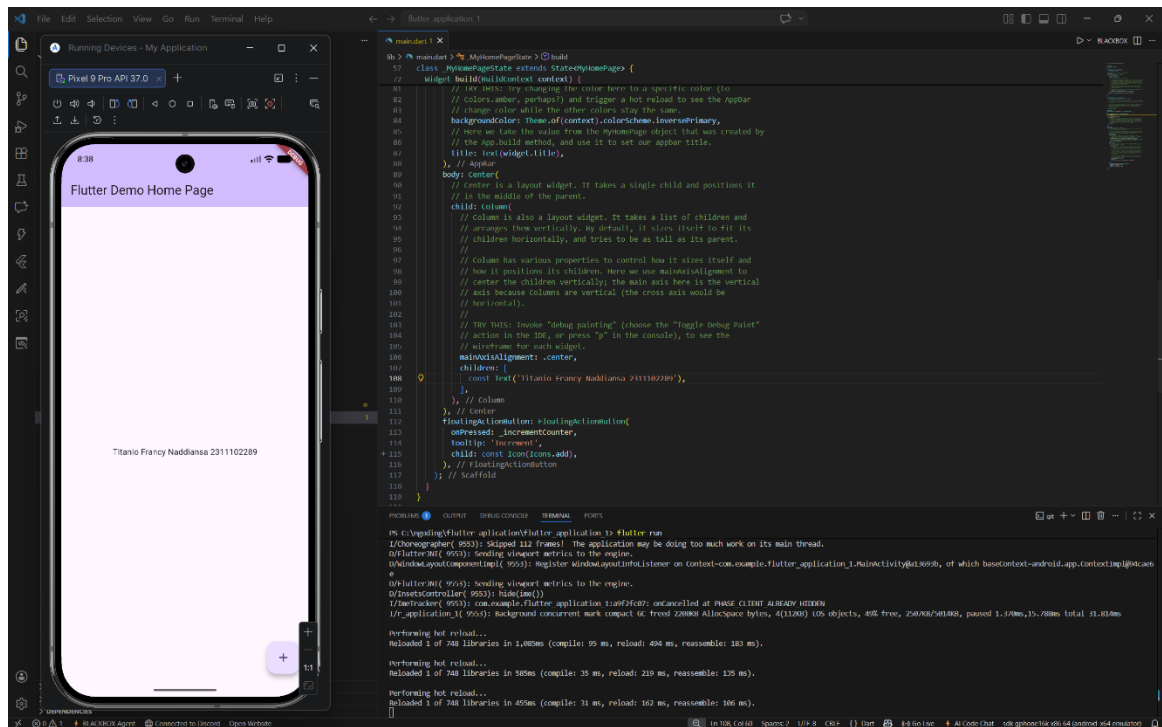


```
class _MyHomePageState extends State<MyHomePage> {  
  @override  
  Widget build(BuildContext context) {  
    return Scaffold(  
      appBar: AppBar(  
        title: Text(widget.title),  
      ),  
      body: Center(  
        child: Text('Hello World'),  
      ),  
    );  
  }  
}
```

48. Jalankan aplikasi dengan menekan F5 atau mengetik perintah flutter run pada terminal.

Penjelasan struktur kode di atas:

- Fungsi main() adalah titik masuk (entry point) dari setiap aplikasi Flutter. Di dalamnya memanggil runApp() yang akan menginisialisasi framework dan menampilkan widget pertama.
- Kelas MyApp merupakan StatelessWidget yang berfungsi sebagai root aplikasi. Widget ini mengembalikan MaterialApp, yang menyediakan berbagai fitur desain Material seperti tema, navigasi, dan lainnya.
- Kelas MyHomePage merupakan StatefulWidget — widget yang dapat berubah tampilannya. State aktualnya disimpan di kelas _MyHomePageState.
- Di dalam _MyHomePageState, method build() mengembalikan widget Scaffold yang menyediakan struktur halaman standar, lengkap dengan AppBar di bagian atas dan body di tengah yang berisi teks 'Hello World'.



BAB 3 – PENGENALAN DART

Tujuan Praktikum

- Mahasiswa mampu memahami konsep dasar dan karakteristik bahasa pemrograman Dart sebagai bahasa utama Flutter.
- Mahasiswa dapat mengimplementasikan sintaks dasar Dart meliputi variabel, kontrol alur program, perulangan, list, dan fungsi.

3.1. Pengenalan Dart

Dart adalah bahasa pemrograman yang dikembangkan oleh Google dan dirancang untuk membangun aplikasi yang berjalan cepat di berbagai platform. Dart memiliki karakteristik yang mirip dengan bahasa C dan Java, sehingga cukup mudah dipelajari bagi yang sudah memiliki dasar pemrograman berorientasi objek. Aturan paling mendasar dalam Dart adalah setiap pernyataan (statement) harus diakhiri dengan tanda titik koma (;).

Untuk dapat mengembangkan aplikasi Flutter, tidak perlu menjadi ahli Dart terlebih dahulu. Pemahaman atas beberapa konsep fundamental berikut sudah cukup untuk memulai.

3.1.1. Variable

Variabel adalah wadah yang digunakan untuk menyimpan data. Di Dart, terdapat tiga cara mendeklarasikan variabel:

- Menggunakan kata kunci var — tipe data akan disimpulkan secara otomatis oleh compiler (type inference).
- Menggunakan type annotation — tipe data dideklarasikan secara eksplisit untuk kejelasan kode.
- Deklarasi multiple variable — mendefinisikan beberapa variabel bertipe sama dalam satu baris.

Sintaks penulisannya:

```
// Menggunakan var (tipe disimpulkan otomatis)
var <variable_name>;
var <name> = <expression>;
```

```
// Menggunakan type annotation (tipe eksplisit)
<type> <variable_name>;
<type> <name> = <expression>;
```

```
// Multiple variable (beberapa variabel satu baris)
<type> <var1, var2, ...varN>;
```

Tipe data primitif yang tersedia di Dart beserta kegunaannya:

- int — menyimpan bilangan bulat, contoh: int umur = 20;
- double — menyimpan bilangan desimal/pecahan, contoh: double tinggi = 170.5;
- String — menyimpan data teks, contoh: String nama = "Titanio";
- bool — menyimpan nilai kebenaran true atau false, contoh: bool aktif = true;

3.1.2. Statement Control

Statement control atau pernyataan kendali digunakan untuk mengatur alur eksekusi program berdasarkan kondisi tertentu. Dart mendukung empat jenis statement control berikut:

IF Statement

Digunakan untuk mengeksekusi blok kode hanya jika kondisi bernilai true.

```
if (condition) {  
    // statements yang dieksekusi jika condition = true  
}
```

IF ELSE Statement

Menyediakan cabang alternatif yang dieksekusi ketika kondisi bernilai false.

```
if (condition) {  
    // dieksekusi jika condition = true  
} else {  
    // dieksekusi jika condition = false  
}
```

IF ELSE IF Statement

Digunakan saat terdapat lebih dari dua kemungkinan kondisi yang perlu diperiksa secara berurutan.

```
if (condition1) {  
    // statement(s)  
}  
else if (condition2) {  
    // statement(s)  
}  
.  
.  
else if (conditionN) {  
    // statement(s)  
}  
else {  
    // statement(s) jika semua kondisi tidak terpenuhi  
}
```

SWITCH CASE Statement

Lebih efisien daripada if-else if apabila terdapat banyak cabang berdasarkan nilai dari satu ekspresi. Setiap case harus diakhiri dengan break untuk menghentikan eksekusi.

```
switch (expression) {  
    case value1: {  
        // statements  
    }  
    break;  
    case value2: {  
        // statements  
    }  
    break;  
    default: {
```

```
    // statements (jika tidak ada case yang cocok)
  }
  break;
}
```

3.1.3. Looping

Looping atau perulangan digunakan untuk mengeksekusi sebuah blok kode secara berulang. Dart menyediakan dua jenis perulangan utama:

For Loop

Gunakan for loop ketika jumlah perulangan sudah diketahui sebelumnya. Struktur for loop terdiri dari nilai awal counter, kondisi berhenti, dan langkah perubahan counter pada setiap iterasi.

```
for (initial_count_value; termination_condition; step) {
  // statements yang dieksekusi setiap iterasi
}

// Contoh: mencetak angka 1 sampai 5
for (int i = 1; i <= 5; i++) {
  print(i);
}
```

While Loop

Gunakan while loop ketika jumlah perulangan tidak diketahui di awal, namun bergantung pada terpenuhinya suatu kondisi. Perulangan akan terus berlanjut selama ekspresi kondisi bernilai true.

```
while (expression) {
  // statements yang dieksekusi selama expression = true
}

// Contoh: berjalan selama nilai belum mencapai 10
int n = 0;
while (n < 10) {
  print(n);
  n++;
}
```

3.1.4. List

List adalah struktur data yang digunakan untuk menyimpan kumpulan nilai dalam satu variabel. Dalam bahasa pemrograman lain konsep ini sering disebut array, namun Dart menamakannya list. Dart menyediakan dua jenis list:

Fixed Length List

Fixed length list memiliki ukuran yang tetap dan ditentukan saat deklarasi. Jumlah elemen tidak dapat ditambah maupun dikurangi setelah list dibuat. Cocok digunakan ketika jumlah data sudah pasti dan tidak akan berubah.

```
// Mendeklarasikan fixed length list dengan ukuran 3
var list_name = new List(initial_size);
```

```
// Mengisi nilai elemen berdasarkan indeks  
list_name[index] = value;
```

```
// Contoh penggunaan:  
var nilaiMahasiswa = new List(3);  
nilaiMahasiswa[0] = 85;  
nilaiMahasiswa[1] = 90;  
nilaiMahasiswa[2] = 78;
```

Growable List

Growable list adalah list yang ukurannya dinamis dan dapat bertambah sesuai kebutuhan. Cocok digunakan ketika jumlah data tidak pasti atau terus bertambah, misalnya daftar item dari database.

```
// Mendeklarasikan growable list (tanpa ukuran tetap)  
var list_name = new List();
```

```
// Menambahkan elemen menggunakan method .add()  
list_name.add(value);
```

```
// Contoh penggunaan:  
var daftarNama = new List();  
daftarNama.add('Titanio');  
daftarNama.add('Francy');  
daftarNama.add('Naddiansa');
```

3.1.5. Fungsi

Fungsi (atau disebut juga metode dalam konteks OOP) adalah blok kode yang dirancang untuk menjalankan tugas tertentu dan dapat dipanggil berkali-kali dari bagian mana saja dalam program. Penggunaan fungsi adalah salah satu cara terbaik untuk menerapkan prinsip DRY (Don't Repeat Yourself) dan meningkatkan keterbacaan kode.

Prinsip-prinsip pemrograman yang umum diterapkan melalui fungsi antara lain:

- SOLID — sekumpulan prinsip desain OOP untuk menghasilkan kode yang bersih dan mudah dikelola.
- KISS (Keep It Simple, Stupid) — setiap fungsi sebaiknya melakukan satu hal saja dengan cara sesederhana mungkin.
- YAGNI (You Aren't Gonna Need It) — hindari menambahkan fungsionalitas yang belum dibutuhkan saat ini.

Semua prinsip tersebut menjunjung tinggi separation of concern — setiap bagian kode memiliki tanggung jawabnya masing-masing dan meminimalkan boilerplate code yang tidak perlu.

Sintaks dasar penulisan fungsi di Dart:

```
// Fungsi dengan return value  
<return_type> <function_name>(<parameters>) {  
  // body fungsi  
  return value; // wajib jika return_type bukan void  
}
```

```
// Contoh fungsi yang mengembalikan nilai
int penjumlahan(int a, int b) {
    return a + b;
}

// Contoh fungsi tanpa return value (void)
void tampilkanPesan(String pesan) {
    print(pesan);
}

// Cara memanggil fungsi di main()
void main() {
    int hasil = penjumlahan(10, 5);
    tampilkanPesan('Hasil penjumlahan: $hasil');
}
```

Dengan mendefinisikan logika di dalam fungsi-fungsi yang terpisah, kode menjadi lebih modular, mudah dibaca, mudah diuji, dan dapat digunakan kembali (reusable) di berbagai bagian program tanpa perlu menulis ulang kode yang sama.